

Atividade de pesquisa - Models

Professor: Irlan Arley Targino Moreira
Almerinda Ventura de Almeida Nascimento
Kellyson Raphael Medeiros da Silva

18 de set.

1.1. O que é um Model no Django e qual problema ele resolve dentro da arquitetura MVT?

No Django, o Model é basicamente a representação dos dados da nossa aplicação em formato de código Python. Cada Model vira uma tabela no banco de dados, e os atributos da classe viram as colunas dessa tabela. O principal problema que ele resolve é centralizar toda a regra e a estrutura desses dados num lugar só. Em vez de ficarmos espalhando comandos de banco ou regras de validação pelas views e templates, deixamos tudo organizado dentro do Model. Com isso, a arquitetura MVT fica bem dividida: o Model cuida das informações, a View gerencia a lógica e o Template lida com a parte visual.

1.2. O que é ORM (Object-Relational Mapping) e como o Django usa esse conceito para evitar escrever SQL manualmente?

O ORM é como se fosse um "tradutor" entre o mundo da orientação a objetos, que seria o Python, e o mundo relacional, no caso as tabelas do banco de dados. Em vez de precisarmos escrever queries SQL complexas manualmente, simplesmente usamos classes e métodos Python. O Django nos dá o gerenciador `objects` em cada Model, permitindo fazer chamadas simples como `.all()`, `.filter()` ou `.create()`. Por trás dos panos, o ORM pega essas chamadas, traduz para o SQL específico do banco de dados que estamos usando e executa pra gente. Isso economiza muito tempo e facilita caso um dia a gente precise trocar de banco de dados.

1.3. Como se declara um Model? Liste pelo menos 5 tipos de campo e explique quando usar cada um

Para criar um Model, criamos uma classe Python que herda de `models.Model`. Cada atributo dessa classe recebe um tipo de campo fornecido pelo Django, de acordo com o dado que

queremos armazenar. Exemplo:

```
2
3
You, 12 minutes ago | author (You)
4 class Livro(models.Model):
5     titulo = models.CharField(max_length=200)
6     autor = models.CharField(max_length=120)
7     ano_publicacao = models.IntegerField()
8     disponivel = models.BooleanField(default=True)
9
You, 12 minutes ago | author (You)
10 class Meta:
11     ordering: list[str] = ["titulo"]
12     verbose_name = "Livro"
13     verbose_name_plural = "Livros"
14
15 def __str__(self):
16     return f"{self.titulo} ({self.ano_publicacao})"
17
```

Alguns dos tipos de campos mais usados no dia a dia:

- CharField: ideal para textos curtos e com limite de caracteres definido, como nomes, títulos ou autores.
- TextField: perfeito para textos longos e sem tamanho fixo, tipo sinopses, comentários ou biografias.
- IntegerField: usado para armazenar números inteiros, como a quantidade de páginas ou o ano de lançamento.
- BooleanField: para campos do tipo verdadeiro/falso (sim/não), como saber se um produto está em estoque.
- DateField / DateTimeField: para trabalhar com datas e horários, muito útil em datas de publicação ou criação de registros.
- ForeignKey: cria um relacionamento de "um para muitos" entre dois Models, relacionando, por exemplo, um livro à sua editora.

1.4. Para que serve a classe Meta dentro de um Model? Cite pelo menos duas opções configuráveis nela.

A classe interna `Meta` serve para definir opções de configuração e metadados da tabela, sem misturar isso com a declaração dos campos. Entre as opções mais conhecidas temos:

- `ordering`: define a ordenação padrão dos registros nas buscas (ex.: `ordering = ['titulo']` ordena por ordem alfabética de título).
- `verbose\_name` e `verbose\_name\_plural`: definem nomes mais amigáveis e legíveis para o Model (no singular e no plural), facilitando a exibição no painel administrativo do Django.

1.5. O que são migrations? Explique a diferença entre `makemigrations` e `migrate`.

As migrations são arquivos de histórico gerados pelo Django para acompanhar todas as alterações estruturais que fazemos nos nossos Models (como criar tabelas, adicionar novos campos ou alterar tipos). Elas garantem que a estrutura do banco evolua junto com o nosso código.

Migrations são arquivos gerados automaticamente pelo Django que registram, passo a passo, as mudanças feitas na estrutura dos Models (criação de tabela, adição de campo, alteração de tipo, etc.). Elas funcionam como um histórico versionado do esquema do banco de dados, permitindo aplicar essas mudanças em qualquer ambiente de forma consistente.

- ``makemigrations``: analisa o arquivo ``models.py`` e cria os arquivos de instrução de migration referentes às mudanças detectadas, mas ainda sem alterar o banco de dados de fato.
- ``migrate``: lê os arquivos de migration que foram gerados e executa as alterações necessárias diretamente na base de dados.

2. Ligando Models às Views

2.1. Como uma view importa e utiliza um Model para buscar dados do banco?

Basta importar a classe do Model no topo do arquivo da View (como ``from .models import Livro``) e depois usar a API do ORM para fazer as consultas necessárias dentro da função da view, como ``Livro.objects.all()``. Depois de resgatar as informações, repassamos esse resultado para o Template através do contexto do ``render()``.

2.2. O que é um QuerySet? Explique a diferença entre ``Model.objects.all()``, ``filter()`` e ``get()``.

Um QuerySet representa uma lista de objetos retornados de uma consulta ao banco de dados. Ele é "preguiçoso" (lazy), o que significa que o Django só vai ao banco buscar os dados quando precisamos usar essa lista de fato (por exemplo, ao iterar em um loop no template).

- ``all()``: busca e retorna todos os registros presentes na tabela do Model.
- ``filter()``: retorna apenas os registros que correspondem a um filtro que informamos (pode retornar vários registros, apenas um ou nenhum).
- ``get()``: busca exatamente um único objeto que atenda ao critério. Se não encontrar nada ou se encontrar mais de um registro, ele lança um erro (``DoesNotExist`` ou ``MultipleObjectsReturned``, respectivamente).

2.3. Como os dados buscados no Model chegam até o Template? Pesquise o dicionário de contexto passado para ``render()``.

Usamos o dicionário de contexto dentro da função ``render(request, 'template.html', contexto)``. As chaves desse dicionário tornam-se variáveis que podemos utilizar diretamente dentro das

tags do template. Por exemplo, se passamos ``{"livros": lista_de_livros}``, conseguimos iterar sobre essa lista dentro do HTML usando ``{% for livro in livros %}``.

2.4. Pesquise o atalho ``get_object_or_404``. Por que ele é preferível a um ``Model.objects.get()`` direto dentro de uma view?

O ``get_object_or_404`` facilita muito porque ele tenta buscar o item com ``.get()``, mas se o item não existir, ele já retorna automaticamente uma página de erro 404 para o usuário (Página não encontrada). Se usássemos apenas o ``.get()`` sem um tratamento de erro, a aplicação geraria uma exceção interna (erro 500), quebrava a requisição e deixaria uma péssima experiência de navegação.

3. Desafio prático — Mini biblioteca

<https://github.com/kellyson71/mini-biblioteca-django>

Mini Biblioteca

Título	Autor	Ano	Disponível
1984	George Orwell	1949	Sim
Dom Casmurro	Machado de Assis	1899	Sim
Grande Sertão: Veredas	Guimarães Rosa	1956	Sim
O Cortiço	Aluísio Azevedo	1890	Não